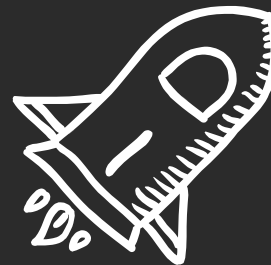




Semana 12

¡Bienvenidos!



Temas de hoy

1 Sistema de Registro

3 Creación de eventos

2 Sistema de Login

1

Funcionalidades básicas: Registro

Como estamos acostumbrados, Django nos provee de facilidades en distintas áreas que suelen ser repetitivas en todos los proyectos como lo es el registro de usuarios de una página web.

Es por ello que veremos cómo registrar un usuario utilizando CBV y FBV en Django.

Registro: Aplicacion blog_auth

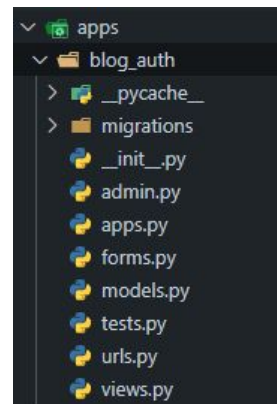
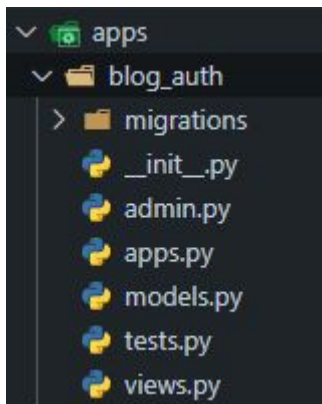
Para comenzar con nuestro sistema de creación y autenticación de los usuarios crearemos una aplicación que se encargue de ello.

Para eso generaremos una nueva aplicación llamada “**blog_auth**”.

Y en esta misma agregaremos dos nuevos archivos:

- forms.py
- urls.py

siguiendo los consejos que aprendimos sobre URLs en la clase anterior.



Registro: Formulario

Formularios

Si bien veremos en profundidad sobre los usos y particularidades de los Formularios en clases futuras, vamos a ver un pequeño pantallazo necesario para poder registrar un usuario.

El archivo **forms.py** nos permite manejar y definir los formularios con los cuales los usuarios van a poder **enviarnos** información al backend y nosotros **manipularlo** a nuestro gusto.

Es por ello que necesitamos de un formulario para poder pedirle al usuario que ingrese su información y nosotros poder guardarlo en nuestra base de datos como un nuevo usuario.

Así se vería un ejemplo de un formulario para el usuario:

Sign Up

It's quick and easy.

First name

Last name

Mobile number or email

New password

Birthdate

Sep

22

2024

Gender

Female

Male

Custom

People who use our service may have uploaded your contact information to Facebook. [Learn more.](#)

By clicking Sign Up, you agree to our [Terms](#), [Privacy Policy](#) and [Cookies Policy](#).
You may receive SMS Notifications from us and can opt out any time.

Sign Up

Facebook nos solicita cierta información **obligatoria** para generarnos una nueva cuenta como lo puede ser el nombre, email, contraseña, fecha de nacimiento, etc.

Y un **botón** para “Registrarse” más que oportuno para finalizar la acción.

Registro: Formulario

Para la creación de un formulario cuyo objetivo sea la creación de un nuevo usuario debemos importar de Django una clase que nos provea de dicha funcionalidad como lo es **UserCreationForm**.

```
from django.contrib.auth.forms import UserCreationForm
```

Para luego poder definir nuestro formulario como una clase y heredar de esta importación sus métodos y atributos:

```
class SignUpForm(UserCreationForm):
```

Es una simple convención pero se recomienda que cuando crean un nuevo formulario, se agregue la palabra Form al final 😊

Gracias a las funcionalidades heredadas, ahora podemos centrarnos en las necesidades que queremos acaparar con nuestro formulario.

Para ello debemos crear un clase **interna** "Meta" el cual se encarga de configurar los metadatos de nuestro formulario.

Por ejemplo:

- **Atributo model:** Nos permite **relacionar** nuestro formulario con el modelo **User** que viene predefinido en Django. El relacionarlos hará que al guardar el formulario se crearán o actualizarán **instancias** de este modelo en la base de datos.
- **Atributo fields:** Indicaremos **que** campos del modelo "User" vamos a querer incluir en nuestro formulario para que el usuario rellene al momento de registrarse.

Dejándonos un código como este:

```
from django.contrib.auth.models import User
```

```
class SignUpForm(UserCreationForm):  
    class Meta:  
        model = User  
        fields = (  
            'username',  
            'email',  
            'password1',  
            'password2'  
        )
```

Importamos el modelo para poder relacionarlo en el atributo model y configuramos para que nuestro formulario requiera los campos username, email, y dos veces la misma contraseña.

Registro: Vista (CBV)

Una vez que nuestro formulario esté listo, ahora podremos trabajar con nuestras vistas para poder procesar las peticiones y trabajar nuestro formulario correctamente.

Como aprendimos anteriormente, el archivo `views` podemos trabajarlo de dos maneras: como vistas basadas en clases (**CBV**) y como vistas basadas en funciones (**FBV**).

Veamos cómo resolver el registro en ambas maneras.

CBV

Como podemos asumir, Django nos tiene preparado una vista genérica de la cual podremos heredar que nos facilita el **manejo de formularios**, la validación de los datos y redirecciones.

Para ello deberemos importar dicha clase:

```
from django.views.generic import FormView
```

Ahora crearemos nuestra clase de registro que lo herede.

```
class SignUpView(FormView):
```

Como bien se dijo antes, ahora tenemos la posibilidad de que pasándole valores a ciertos atributos heredados, Django sea capaz de relacionar y resolver el registro completamente.

```
class SignUpView(FormView):  
    template_name = "registration/register.html"  
    form_class = SignUpForm  
    success_url = reverse_lazy("apps.blog_auth:login")
```

- **template_name** : definimos **cuál** será el template o plantilla en el que deba renderizar el formulario. *(En un futuro profundizaremos más en plantillas pero lo utilizaremos para el ejemplo 😊)*
- **form_class** : definimos **cuál** formulario será el que usaremos en la vista y rendericemos en el template.
- **success_url** : especificamos a qué url queremos redireccionar nuestra página web cuando el registro se realice correctamente.

Registro: Vista (CBV)

Por último **validaremos** el formulario previo a crear el usuario (generar el nuevo registro en nuestra base de datos).

Para ello utilizaremos un método heredado de `FormView` llamado `form_valid()` el cual se ejecuta una vez que todos los datos del formulario hayan pasado las validaciones:

```
def form_valid(self, form):
```

El parámetro `form` recibirá automáticamente como argumento la información de la petición (los datos del formulario) y luego guardaremos los datos de dicho formulario.

Por último, llamaremos al método `form_valid()` pero de la clase padre (`FormView`) que se encargará de redirigir al usuario a la ruta que definimos en el atributo `success_url`.

```
def form_valid(self, form):  
    """Verificamos que los datos sean válidos y los guardamos"""  
    form.save()  
    return super().form_valid(form)
```

Dandonos como resultado una vista como esta:

```
class SignUpView(FormView):  
    template_name = "registration/register.html"  
    form_class = SignUpForm  
    success_url = reverse_lazy("apps.blog_auth:login")  
  
    def form_valid(self, form):  
        """Verificamos que los datos sean válidos y los guardamos"""  
        form.save()  
        return super().form_valid(form)
```


Registro: Vista (FBV)

FBV

A diferencia de las vistas basadas en clases, cuando desarrollamos con funciones **no** contamos con la posibilidad de heredar funcionalidades que nos faciliten algunas tareas, por lo tanto nos toca realizar algunos pasos extras para lograr el mismo resultado.

- **Métodos HTTP**

Cada solicitud HTTP enviada por el usuario posee un **método** que varía dependiendo de la acción que desea realizar. En total existen **9 métodos diferentes** que se utilizan en diferentes situaciones pero nos centraremos en el **método GET** y el **método POST**.

Cuando el usuario hace una petición o solicitud que requiera **recuperar datos del servidor**, como lo puede ser una lista de productos en una tienda online, la lista de tus seguidores en una red social o cuando ingresas a tu perfil, son datos ya almacenados que deseamos simplemente **consultar**. Cada una de esas solicitudes utilizan el método **GET** ya que **no** modifica los datos, y podemos repetir la misma petición una y otra vez sin cambiar el estado del servidor.

Por otro lado, cuando un usuario desea **enviar datos al servidor**, ya sea para registrarse en un página, comentar en un video o publicar un tweet, es el método **POST** el que se encarga de comunicarse con el servidor y dar a entender al mismo de que debe procesar esos datos y devolver una respuesta.

Registro: Vista (FBV)

Ahora que entendemos la diferencia entre los métodos **GET** y **POST** vayamos a nuestra vista de registro.

Lo primero que debemos recordar es que las funciones deben tener obligatoriamente un parámetro **request** en donde estaremos recibiendo la solicitud del usuario:

```
def register_view(request):
```

Una vez definida nuestra función, haremos una primera validación donde filtraremos el método de nuestra solicitud request, debido a que al momento de registrar un nuevo usuario, a nosotros solamente nos interesa las solicitudes con método POST que nos envían información, y no cualquier otro método:

```
if request.method == "POST":
```

En caso de que el método de la solicitud sea POST y la condición sea verdadera, crearemos una instancia de nuestro formulario de registro que hicimos al comienzo pasándole como argumento todos los valores que recuperamos del formulario rellenado:

```
def register_view(request):  
    if request.method == "POST":  
        form = SignUpForm(request.POST)
```

Y repetimos el proceso de validación del formulario al igual que en la vista CBV con algunas variaciones:

```
def register_view(request):  
    if request.method == "POST":  
        form = SignUpForm(request.POST)  
        if form.is_valid():  
            form.save()  
            return redirect("apps.blog_auth:login")
```

En CBV nosotros heredamos una método "form_valid()" pero ...

Registro: Vista (FBV)

... al trabajar con FBV, la instancia del formulario nos provee una función llamada `is_valid()` que nos devuelve “true” en caso de que los datos se validen correctamente.

Por lo tanto en caso de ser verdadero, **guardaremos** el formulario y **redireccionaremos** al usuario a una ruta a elección.

Este algoritmo se ejecuta en caso de que el método de la solicitud sea **POST**, pero, ¿y si no lo es?

Si no lo es, entonces el usuario no está enviando información, si no que simplemente entró a la página para registrarse, por lo tanto debemos devolverle como respuesta el formulario **vacío** para que pueda rellenar y enviar esa información.

Eso lo logramos creando una instancia de nuestro formulario, y pasándolo como **contexto** (3er argumento) a nuestra función `render()`:

```
def register_view(request):
    if request.method == "POST":
        form = SignUpForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("apps.blog_auth:login")
    else:
        form = SignUpForm()
        return render(request, "registration/register.html", {'form': form})
```

En conclusión, si el método no es POST, se crea una instancia del formulario guardada en “**form**” y luego al momento de retornar la función render, pasamos un diccionario cuya clave-valor sea la instancia recién creada.

Registro: URLs

CBV

```
from django.urls import path
from .views import SignUpView

app_name = "apps.blog_auth"

urlpatterns = [
    path("register/", SignUpView.as_view(), name="register"),
]
```

Dentro de nuestra lista **urlpatterns** agregamos nuestra función path con los argumentos:

- **"register/"** : como la dirección de la URL a la cual debe ingresar el usuario
- **SignUpView.as_view()** : Importamos la vista creada en views.py, y al ser una clase lo que importamos, debemos utilizar **.as_view()** para que Django lo convierta en una función capaz de recibir solicitudes HTTP.

FBV

```
from django.urls import path
from .views import register_view

app_name = "apps.blog_auth"

urlpatterns = [
    path("register/", register_view, name="register")
]
```

Dentro de nuestra lista **urlpatterns** agregamos nuestra función path con los argumentos:

- **"register/"** : como la dirección de la URL a la cual debe ingresar el usuario
- **register_view** : Importamos la vista que hicimos en views.py y la pasamos como 2do argumento
- **name="register"**: Asignamos un nombre para facilitar y separar lo que es la ruta de acceso con la URL.

2

Funcionalidades básicas: Login

Ya fuimos capaces de familiarizarnos más con las facilidades que nos provee este Framework a la hora de enfrentarnos a mecanismos que suelen ser complejos como el registro de un usuario. Ya te imaginarás lo que nos tiene preparado Django ahora que queremos ingresar con un usuario

ya creado 😊

Login: Vista (CBV)

A diferencia del registro de un usuario, **no** vamos a requerir crear un nuevo formulario debido a que Django nos proporciona uno que cumple con lo esperado.

Por lo tanto, veamos cómo resolveremos en las vistas:

CBV

Django posee vistas de autenticación en las cuales podemos encontrar **LoginView** que será la que nos proporcionará todos los mecanismos de autenticación del usuario. Para ello debemos importarla:

```
from django.contrib.auth import views as auth_views
```

Importamos desde **django.contrib.auth** todas sus views y las renombramos como **auth_views** para evitar conflictos o confusiones con otras posibles vistas que se quieran implementar.

Una vez importada la vista de Django, crearemos nuestra clase **Login** que herede de LoginView y definiremos como **único** atributo el template/plantilla que ocupará:

```
class Login(auth_views.LoginView):  
    template_name = "registration/login.html"
```

Si bien parece que no estamos definiendo nada, y que no debería funcionar nuestro código, la vista **LoginView** de Django es internamente muy completa.

Esta vista de la cual estamos heredando por detrás está:

- Utilizando un formulario predefinido por Django (AuthenticationForm) el cual define los campos username y password como los campos del formulario.
- Valida los datos con la base de datos verificando que exista el usuario y verifica que las credenciales sean válidas.
- Establece un token de sesión para el usuario.
- Y por último redirecciona al usuario en caso de estar todo correctamente

Login: Vista (FBV)

FBV

Si decidimos tratar el Login con FBV, debemos desarrollar cada uno de los puntos que nos resuelve **LoginView** en CBV. Partiendo de los conceptos aprendidos cuando creamos la vista de registro: debemos definir la función con nuestro parámetro request y filtrar según el método de la solicitud:

```
def login_view(request):
    if request.method == 'POST':
```

En el caso de que sea **verdadera** dicha condición, vamos a obtener de la solicitud POST los campos que nos enviaron mediante el formulario de la siguiente manera:

- **request.POST** obtiene la información enviada por dicho método.
- **.get("campo")** busca un campo en los datos enviados en la solicitud con el nombre pasado por parámetro.

```
username = request.POST.get("username")
password = request.POST.get("password")
```

Una vez que tenemos las credenciales del formulario, los pasamos como argumentos por la función **authenticate()** con el cual Django va a verificar si el username y el password coinciden con algún usuario registrado en la base de datos. En caso de que sean válidas, nos devuelve un **objeto "User"** que lo almacenamos en la variable "user". En caso de que sean incorrectas, devuelve **None**.

```
from django.contrib.auth import authenticate, login

user = authenticate(request, username=username, password=password)
```

Login: Vista (FBV)

Si en la variable **user** tenemos almacenado un **objeto** (una instancia de la clase User), es decir, es lo contrario a None, entonces podemos proceder a hacer el “**login**” del usuario y redireccionar una vez finalizado el ingreso.

La función **login()** que la importamos junto a la de *authenticate()*, lo que hará es establecer una **sesión** al usuario que se autenticó. Una sesión le permite al usuario permanecer “**logueado**” o conectado en el sitio durante el tiempo que dure la sesión.

```
if user is not None:
    login(request, user)
    return redirect("apps.blog_auth:login")
```

En el caso de que la variable **user** tenga almacenado el valor “**None**”, entonces deberemos emitir un mensaje de error avisando que las credenciales no son válidas.

```
from django.contrib import messages
```

```
if user is not None:
    login(request, user)
    return redirect("apps.blog_auth:home")
else:
    messages.error(request, "Credenciales incorrectas")
```


Login: Vista (FBV)

```
def login_view(request):
    if request.method == 'POST':
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            return redirect("apps.blog_auth:login")
        else:
            messages.error(request, "Credenciales incorrectas")

    return render(request, "registration/login.html")
```

Y por último retornamos la función **render()** mostrando el formulario en el template **"login.html"**

Login: URLs

CBV

```
from django.urls import path
from .views import SignUpView
from django.contrib.auth import views as auth_views

app_name = "apps.blog_auth"

urlpatterns = [
    path("register/", SignUpView.as_view(), name="register"),
    path("login/", auth_views.LoginView.as_view(), name="login")
]
```

Dentro de nuestra lista **urlpatterns** agregamos nuestra función path con los argumentos:

- **"login/"** : como la dirección de la URL a la cual debe ingresar el usuario
- **auth_views.LoginView.as_view()** : Importamos la vista de LoginView proveniente de auth_views, y al ser una clase la importación, debemos utilizar **.as_view()** para que Django lo convierta en una función capaz de recibir solicitudes HTTP.

FBV

```
from django.urls import path
from .views import login_view, register_view

app_name = "apps.blog_auth"

urlpatterns = [
    path("login/", login_view, name="login"),
    path("register/", register_view, name="register")
]
```

Dentro de nuestra lista **urlpatterns** agregamos nuestra función path con los argumentos:

- **"login/"** : como la dirección de la URL a la cual debe ingresar el usuario
- **login_view** : Importamos la vista que hicimos en views.py y la pasamos como 2do argumento.
- **name="login"**: Asignamos un nombre para facilitar y separar lo que es la ruta de acceso con la URL.

2

Funcionalidades básicas: Creación de eventos

Aprendimos a utilizar la ORM y a importar e implementar las vistas genéricas que nos provee Django en clases anteriores.

Ahora veamos cómo podemos desarrollar una aplicación genérica que se adapte a nuestras necesidades con todo lo aprendido.

Creación de eventos: Forms y Models

Cuando hablamos de “Creación de eventos” nos referimos por eventos a los distintos **objetivos** que perseguirán los sitios webs que desarrollen. Por ejemplo, en el caso del proyecto final, deben desarrollar un sistema de “**noticias**” con todas sus funcionalidades, por lo tanto dicho sistema será su evento en cuestión.

Para empezar a trabajar las diferentes vistas con sus respectivas rutas, primero debemos definir el **modelo** de la aplicación y el **formulario** que precisamos para agregar registros a nuestra base de datos.

Simple aclaración: Como bien sabemos, el archivo **forms.py** y **models.py** son independientes de **cómo** se desarrollen las vistas, es decir que, si las vistas las desarrollamos como FBV o CBV, los modelos y formularios no se ven afectados, sin embargo las **rutas si**.

Por lo tanto, a lo largo del ejemplo, siempre mantendremos estos dos archivos sin alteraciones.

models.py

```
from django.db import models

class Eventos(models.Model):
    titulo = models.CharField(max_length=200)
    descripcion = models.TextField()
    fecha = models.DateField()
    ubicacion = models.CharField(max_length=200)

    def __str__(self):
        return self.titulo
```

Nuestro modelo contendrá los campos titulo, descripcion, fecha y ubicacion. Nótese que fecha **no** se le auto-genera un valor como estamos acostumbrados debido a que queremos ser capaces de establecer manualmente la fecha en el que se realizará el evento

Creación de eventos: Forms y Models

forms.py

```
class EventoForm(forms.ModelForm):  
    class Meta:  
        model = Eventos  
        fields = ['titulo', 'descripcion', 'fecha', 'ubicacion']  
        widgets = {  
            'fecha': forms.DateInput(attrs={'type': 'date'}),  
        }  
    
```



En nuestra clase Meta definiremos el **modelo** con el cual va a interactuar nuestro formulario, y los **campos** que vamos a permitir que completen los usuarios en el formulario que corresponden con los campos del modelo definido.

Además agregamos un **widget** que define cómo debe ser el formato del campo “**fecha**” en el formulario. En lugar de que te permita agregar una cadena de texto, te mostrará un almanaque para seleccionar.

admin.py

```
from django.contrib import admin  
from .models import Eventos  
  
admin.site.register(Eventos)
```



Recordemos de registrar nuestro **modelo** al Administrador de Django en el archivo **admin.py** para poder gestionarlo desde dicha herramienta

Creación de eventos: Vistas (CBV)

Para nuestro evento debemos desarrollar un **CRUD completo** que resuelva los aspectos principales y generales de una aplicación.

CBV

CREATE VIEW

```
from django.views.generic import CreateView
from .models import Eventos
from .forms import EventoForm
```

Primero importamos los módulos que necesitaremos para el desarrollo de esta vista como lo son:

- Nuestro modelo → Eventos
- Nuestro formulario → EventoForm
- La vista predefinida de Django → CreateView

```
class CrearEventoView(CreateView):
    model = Eventos
    form_class = EventoForm
    template_name = 'eventos/form_evento.html'
    success_url = reverse_lazy('lista_eventos')
```

- Asignamos a nuestro modelo “**Eventos**” como valor del atributo “**model**” con el objetivo de que esta vista se vincule con la tabla Eventos de nuestra base de datos.
- En el atributo **form_class** definimos nuestro formulario **EventoForm** para que nos provea de la estructura correcta al momento de generarse el formulario.
- En **template_name** definimos el template que utilizará.
- Y por último en **success_url** debemos otorgar como valor la acción que debe suceder en caso de que todo haya resultado **correctamente**. En este caso, con la ayuda de la función **reverse_lazy()** volvemos a la página inicial.

Creación de eventos: Vistas (CBV)

CBV

READ VIEWs

```
from django.views.generic import (
    CreateView,
    ListView,
    DetailView
)
```

Importamos las vistas genéricas de Django:

- **ListView** : Esta vista automáticamente realiza la lógica necesaria para obtener **todos** los objetos del modelo que especifiquemos y los envía al template.

- **DetailView** : Por otro lado, esta vista recibe mediante su URL la **id/pk** de **uno** de los objetos y realiza la lógica para buscar ese objeto en particular y enviar su información al template dándonos como resultado el detalle de un solo objeto en específico.

```
class DetalleEventoView(DetailView):
    model = Eventos
    context_object_name = "evento"
    template_name = "eventos/detalle_evento.html"
```

Al igual que nuestra vista anterior, definimos el modelo con el cual hará las **consultas** necesarias y el template que **renderizará** cuando tenga la información.

Por otro lado, utilizamos el atributo **context_object_name** para asignar un **nombre al contexto** con el cual manipularemos la información en el template.

Creación de eventos: Vistas (CBV)

```
class ListarEventosView(ListView):
    model = Eventos
    template_name = "eventos/lista_eventos.html"
    context_object_name = "eventos"
    paginate_by = 4

    def get_queryset(self):
        query = self.request.GET.get("titulo")
        queryset = super().get_queryset()

        if query:
            queryset = queryset.filter(titulo__icontains=query)

        return queryset.order_by("titulo")
```

La vista genérica **ListView** nos permite agregar un atributo **paginate_by** el cual nos limita la cantidad de objetos que mostrará por página, en este caso 4. Por lo tanto, suponiendo que tenemos 6 eventos **guardados** en nuestra base de datos, solo se visualizarán 4 y deberemos **movernos** a la siguiente página para ver los 2 restantes.

El mecanismo de paginación para movernos entre páginas lo resolveremos desde los templates más adelante 🤖

Por otro lado, les resultará familiar este método de la clase pasada.

Reutilizamos el método **get_queryset()** el cual nos permitirá a nosotros realizar un **“buscador”** desde el front(templates). Como podemos ver en este ejemplo, la búsqueda se hará según el título del evento:

- `queryset.filter(titulo__icontains=query)`

Creación de eventos: Vistas (CBV)

CBV

UPDATE VIEW

```
from django.views.generic import (  
    CreateView,  
    ListView,  
    DetailView,  
    UpdateView,  
)
```

```
class ActualizarEventoView(UpdateView):  
    model = Eventos  
    form_class = EventoForm  
    template_name = "eventos/form_evento.html"  
    success_url = reverse_lazy("lista_eventos")
```

Importamos la vista genérica **UpdateView** cuyas funcionalidades son muy similares a las que vimos en **CreateView** con la diferencia de que el formulario que nos generará vendrá ya **poblado** con la información del objeto que deseamos modificar/actualizar.

Esta vista también requiere que el **id/pk** del objeto sea transferido mediante su **URL** ya que primero debe buscar el objeto que deseamos modificar y luego nos da la posibilidad de hacerlo.

Además maneja la **validación automática** en el caso de que algún dato no corresponda.

Creación de eventos: Vistas (CBV)

CBV

DELETE VIEW

```
from django.views.generic import (
    CreateView,
    ListView,
    DetailView,
    DetailView,
    DeleteView,
)
```

```
class EliminarEventoView(DeleteView):
    model = Eventos
    template_name = "eventos/confirmacion_eliminacion.html"
    success_url = reverse_lazy("lista_eventos")
```

Importamos la vista genérica **DeleteView** que nos permitirá eliminar un objeto de nuestra base de datos.

Al igual que las vistas anteriores, debemos vincular nuestro modelo. Sin embargo el template cumple un rol diferente al resto, debido a que en él habrá un formulario de confirmación previo a eliminar el objeto.

Es decir, cuando un usuario quiera eliminar un evento, lo llevará a un template donde deberá **reafirmar su decisión** y **aceptar** que desea eliminarlo.

En caso de que todo resulte correctamente, el atributo **success_url** lo redirigirá a la página principal de los eventos.

Creación de eventos: URLs (CBV)

CBV

URLS

```
from django.urls import path
from .views import ListarEventosView, DetalleEventoView, CrearEventoView, ActualizarEventoView, EliminarEventoView

urlpatterns = [
    path('', ListarEventosView.as_view(), name='lista_eventos'),
    path('<int:pk>/', DetalleEventoView.as_view(), name='detalle_evento'),
    path('crear/', CrearEventoView.as_view(), name='crear_evento'),
    path('editar/<int:pk>/', ActualizarEventoView.as_view(), name='actualizar_evento'),
    path('eliminar/<int:pk>/', EliminarEventoView.as_view(), name='eliminar_evento'),
]
```

Importamos todo el **CRUD** de vistas que realizamos y le asignamos una **ruta** a cada una.

Debido a que nosotros redirigimos estas rutas desde el archivo urls.py de la aplicación principal, debemos asumir que previo a cada “path” le antecede “**eventos/**” haciendo que las rutas completas sean: “**eventos/crear/**”, “**eventos/editar/<int:pk>/**”, etc.

Por otro lado, nótese que las vistas en las que habíamos aclarado anteriormente que necesitaban del **id/pk** de un objeto para funcionar correctamente, tienen su ruta preparada para recibirlo como lo son las rutas de **DetalleEvento**, **ActualizarEvento** y **EliminarEvento**.

Creación de eventos: Vistas (FBV)

Ahora vamos a replicar las mismas vistas pero utilizando vistas basadas en funciones utilizando los mismos archivos forms.py y models.py.

FBV

CREATE VIEW

```
def crear_evento(request):  
    if request.method == "POST":  
        form = EventoForm(request.POST)  
        if form.is_valid():  
            form.save()  
            return redirect("lista_eventos")  
        else:  
            form = EventoForm()  
    return render(request, "eventos/form_evento.html", {"form": form})
```

Definimos nuestra función recibiendo la **request** por **parámetro obligatoriamente**.

Luego debemos hacer una primera verificación de que la solicitud sea haya sido enviada por **POST** debido a que si no hay información enviada, nos produciría **error** al intentar guardar el formulario con la request vacía.

En caso de ser **verdadera** la condición, crearemos una instancia del formulario **EventoForm** con la información que recibimos de la request y la guardamos en la variable **form**.

Realizamos una última verificación de que el form sea correcto utilizando **is_valid()** y en el caso de que sea **verdadero**, lo guardaremos en base de datos utilizando **.save()** y redirigimos a la página principal de los eventos.

Si **no** fue enviada por **POST** la solicitud, simplemente creamos una instancia del formulario vacío y la enviamos como **contexto** para que sea renderizada y el usuario pueda rellenarla y enviarla.

Creación de eventos: Vistas (FBV)

FBV

READ VIEWS

```
from django.core.paginator import Paginator
```

```
def listar_evento(request):
    eventos = Eventos.objects.all()

    query = request.GET.get('titulo', '')
    if query:
        eventos = eventos.filter(titulo__icontains=query)

    paginator = Paginator(eventos, 4)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    return render(request, 'eventos/lista_eventos.html', {'page_obj': page_obj, 'query': query})
```

La función `listar_evento()` es el directo proporcional a `ListView()` en **CBV**, por lo tanto nuestros objetivos son:

- Mostrar todos los objetos del modelo.
- Poder buscar un objeto por su nombre desde el front.
- Limitar la cantidad de objetos que se mostraran y dejar listo para realizar la paginación desde el front.

Primero obtenemos con la ORM todos los objetos que posee nuestro modelo Eventos.

Luego buscamos si existe algún string en la **query** con la cual debemos filtrar nuestros eventos. Si encuentra, lo filtra.

Para desarrollar la lógica relacionada a la **paginación**, importamos “**Paginator**” el cual requiere de dos argumentos: la información total que mostrará y la cantidad límite de objetos por página (4).

Siguiendo la misma lógica del *buscador*, si decidimos cambiar de página, se agregará al query el número de página al que deseamos ir, y recibimos ese valor guardándolo en **page_number**. Por último lo pasamos como argumento al método **get_page()** para obtener los objetos que corresponden a esa página, y lo guardamos en el contexto para luego **renderizarlo**.

Creación de eventos: Vistas (FBV)

FBV

READ VIEWs

```
def detalle_evento(request, id):  
    try:  
        evento = Eventos.objects.get(id=id)  
    except Eventos.DoesNotExist:  
        return HttpResponse("Evento no encontrado", status=404)  
  
    return render(request, "eventos/detalle_evento.html", {"evento": evento})
```

La función **detalle_evento()** resuelve el mismo objetivo que **DetailView** en **CBV**, por lo tanto además de recibir la request por parámetro, también debe recibir el **id/pk** con el cual buscaremos el objeto en específico.

Requerimos utilizar **try/except** para manipular la función **get()** de la ORM debido a que si no existe el id, nos devuelve una excepción y debemos ser capaces de recibirla.

En caso de que sí encuentre el objeto con el **id/pk**, pasaremos el evento en el **contexto** y **renderizamos** el template.

Creación de eventos: Vistas (FBV)

FBV

UPDATE VIEW

```
def actualizar_evento(request, id):
    try:
        evento = Eventos.objects.get(id=id)
    except Eventos.DoesNotExist:
        return HttpResponse("Evento no encontrado", status=404)

    if request.method == "POST":
        form = EventoForm(request.POST, instance=evento)
        if form.is_valid():
            form.save()
            return redirect("lista_eventos", id=evento.id)
    else:
        form = EventoForm(instance=evento)

    return render(request, "eventos/form_evento.html", {'form': form, 'evento': evento})
```

La función `actualizar_evento()` es el directo proporcional a `UpdateView` en CBV.

Al igual que en `detalle_evento()`, vamos a requerir por parámetro la `request` y el `id/pk`, y además utilizamos nuevamente...

...la estructura `try/except` para manipular controladamente la **ORM** al intentar obtener al objeto mediante su `id`.

Luego verificamos si la request fue enviada por **POST** en caso de que el formulario haya sido enviado con información:

- Si es verdadero, creamos una instancia de nuestro form con la nueva información (`request.POST`) y vinculamos esta nueva instancia con el objeto que deseamos que se modifique en la base de datos (`instance=evento`). Por último, validamos el form, y lo guardamos en base de datos.
- Si es falso, creamos simplemente la instancia del formulario y lo rellenamos de la información que ya tenía en la base de datos (`instance=evento`) para luego pasarlo por **contexto** y **renderizarlo**. Así el usuario podrá ver la información actual y modificarla.

Creación de eventos: Vistas (FBV)

FBV

DELETE VIEW

```
def eliminar_evento(request, id):  
    try:  
        evento = Eventos.objects.get(id=id)  
    except Eventos.DoesNotExist:  
        return HttpResponse("Evento no encontrado", status=404)  
  
    if request.method == "POST":  
        evento.delete()  
        return redirect("lista_eventos")  
  
    return render(request, "eventos/confirmacion Eliminacion.html", {"evento": evento})
```

La lógica **base** se vuelve repetitiva debido a que requerimos de los mismos valores y analizarlos de la misma manera para su correcto funcionamiento.

Nuevamente hacemos la verificación del método con el cual se envía la request ya que debemos **renderizar** el template que cuestione al usuario si está seguro de eliminar el objeto en cuestión. Si el usuario **confirma** la eliminación, se enviará por **POST** la solicitud, y se ejecutará el **método** **“.delete()”** de la **ORM** el cual se encargará de quitar dicho evento encontrado por **id/pk** de la base de datos.

Creación de eventos: URLs (FBV)

FBV

URLS

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.listar_evento, name='lista_eventos'),
    path('<int:id>/', views.detalle_evento, name='detalle_eventos'),
    path('crear/', views.crear_evento, name='crear_evento'),
    path('editar/<int:id>/', views.actualizar_evento, name='actualizar_evento'),
    path('eliminar/<int:id>/', views.eliminar_evento, name='eliminar_evento'),
]
```



**¡Nos vemos
En la próxima
clase!**

